

Who Gets Power First? A Classification Model

Quick Start Orange County: AI Design Contest — Week 3

Your regression model predicts *how much* power the grid needs. But prediction alone is not enough when supply runs short.

During an emergency — a hurricane, a heat wave, a mass casualty event — your smart city cannot power everyone at full capacity. Someone has to wait. The question is: **who?**

That is a classification problem. You are not predicting a number. You are assigning each power user to a **priority tier** — a category that determines their place in the restoration queue when the grid is stressed.

Why classification? Because the question is *which group does this belong to?* not *how much?* The model learns to sort power users based on their consumption patterns — the same patterns you drew in Week 1.

The stakes are real. A hospital miscategorized as Non-Critical during an outage is a life-safety failure. A school miscategorized as Critical wastes scarce power that a fire station needed. Every classification error has a consequence.

By the end of this notebook your team will have:

- Defined and justified your own priority tier labels
- Engineered features that capture what makes a building critical
- Trained and evaluated a decision tree classifier
- Interpreted the confusion matrix as a real-world risk assessment
- Written a reflection connecting misclassification to human consequences

Keep your Quick Reference sheet open — the power restoration priority order is the framework for your labeling decisions.

[INSTRUCTOR NOTE: Insert Classification companion reading here]

Setup

```
In [1]: import os
import pandas as pd
import numpy as np
```

```

import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree
from sklearn.metrics import (
    classification_report, confusion_matrix,
    ConfusionMatrixDisplay, accuracy_score
)

plt.rcParams['figure.figsize'] = (12, 5)
plt.rcParams['axes.grid'] = True
plt.rcParams['grid.alpha'] = 0.3
plt.rcParams['font.size'] = 11

# Color palette – one color per priority tier, consistent throughout
TIER_COLORS = {
    'Critical': '#E53935', # red
    'Flexible': '#FB8C00', # orange
    'Non_Critical': '#43A047', # green
}

print(f"Looking for data in: {os.getcwd()}")
print("Libraries loaded successfully")

```

Looking for data in: c:\Users\TAGZ1\Desktop\AI Smart Grid\week 3\Task 8 Notebook Files
 Libraries loaded successfully

```

In [2]: # Load the classification dataset
df = pd.read_csv('smart_city_classification.csv')

print(f"Shape: {df.shape[0]:,} rows x {df.shape[1]} columns")
print(f"\nColumns: {df.columns.tolist()}")
print(f"\nBuilding types in dataset:")
print(sorted(df['building_type'].unique()))
print(f"\nFirst 5 rows:")
df.head()

```

Shape: 3,800 rows x 12 columns

Columns: ['building_type', 'hour', 'is_business_hours', 'energy_kWh', 'pct_of_daily_max', 'normalized_demand', 'is_high_demand', 'global_mean_kWh', 'global_max_kWh', 'global_std_kWh', 'priority_tier', 'tier_encoded']

Building types in dataset:

['bus_station', 'clinic', 'college', 'commercial_office', 'fast_food', 'fire_station', 'fitness_center', 'gas_station', 'grocery', 'hospital', 'hotel', 'k12_school', 'library', 'movie_theater', 'police_station', 'post_office', 'restaurant', 'shopping_mall', 'warehouse']

First 5 rows:

```
Out[2]:
```

	building_type	hour	is_business_hours	energy_kWh	pct_of_daily_max	normalized_de
0	hospital	0.25	0	333.561954	0.481135	-1.7
1	restaurant	0.25	0	5.533698	0.256769	-1.3
2	clinic	0.25	0	1.010245	0.395381	-1.9
3	hotel	0.25	0	19.222715	0.594971	-0.8
4	commercial_office	0.25	0	8.531045	0.452878	-2.3

Part 1: Understanding the Dataset

This dataset is structured differently from the regression dataset. Each row is one building type at one time interval — 19 building types × 200 time steps = 3,800 rows.

The features describe each building's energy consumption behavior. The target is `priority_tier` — the label your model will learn to predict.

Before anything else — inspect the data.

```
In [3]: # Dataset structure
print("Data types:")
print(df.dtypes)
print(f"\nMissing values: {df.isnull().sum().sum()}")
print(f"\nLabel distribution:")
print(df['priority_tier'].value_counts())
print(f"\nBuildings per tier:")
print(df.groupby('priority_tier')['building_type'].nunique())
```

```
Data types:
building_type      str
hour               float64
is_business_hours  int64
energy_kWh         float64
pct_of_daily_max   float64
normalized_demand  float64
is_high_demand     int64
global_mean_kWh    float64
global_max_kWh     float64
global_std_kWh     float64
priority_tier      str
tier_encoded       int64
dtype: object
```

```
Missing values: 0
```

```
Label distribution:
priority_tier
Non_Critical    2000
Flexible        1000
Critical         800
Name: count, dtype: int64
```

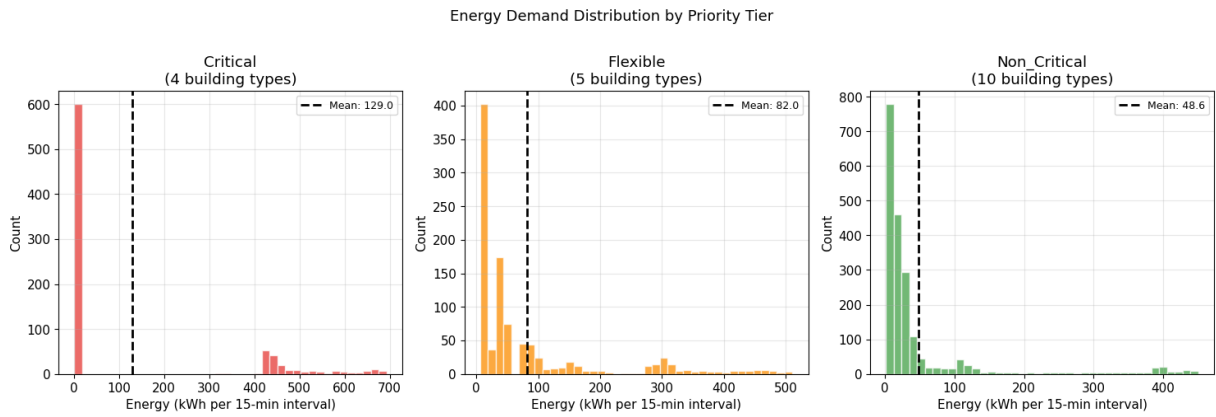
```
Buildings per tier:
priority_tier
Critical         4
Flexible         5
Non_Critical     10
Name: building_type, dtype: int64
```

```
In [4]: # Visualize energy demand by priority tier
# This shows what the model will use to distinguish tiers

fig, axes = plt.subplots(1, 3, figsize=(15, 5))

for i, (tier, color) in enumerate(TIER_COLORS.items()):
    tier_data = df[df['priority_tier'] == tier]
    axes[i].hist(tier_data['energy_kWh'], bins=40,
                 color=color, alpha=0.75, edgecolor='white')
    axes[i].set_title(f'{tier}\n({tier_data["building_type"].nunique()}) building ty
    axes[i].set_xlabel('Energy (kWh per 15-min interval)')
    axes[i].set_ylabel('Count')
    axes[i].axvline(tier_data['energy_kWh'].mean(),
                    color='black', linestyle='--', linewidth=2,
                    label=f'Mean: {tier_data["energy_kWh"].mean():.1f}')
    axes[i].legend(fontsize=9)

plt.suptitle('Energy Demand Distribution by Priority Tier', fontsize=13, y=1.01)
plt.tight_layout()
plt.show()
```



```
In [5]: # Per-building summary – what does each building type look like?
building_summary = df.groupby(['building_type', 'priority_tier']).agg(
    mean_kWh=('energy_kWh', 'mean'),
    max_kWh=('energy_kWh', 'max'),
    std_kWh=('energy_kWh', 'std'),
    variability=('energy_kWh', lambda x: x.std() / x.mean())
).round(2).reset_index()

# Sort by mean demand descending
building_summary = building_summary.sort_values('mean_kWh', ascending=False)
print("Building types by energy demand:")
print(building_summary[['building_type', 'priority_tier', 'mean_kWh', 'std_kWh', 'varia
```

Building types by energy demand:

building_type	priority_tier	mean_kWh	std_kWh	variability
hospital	Critical	492.12	88.30	0.18
shopping_mall	Non_Critical	287.11	118.92	0.41
college	Flexible	279.88	113.40	0.41
grocery	Non_Critical	72.33	26.35	0.36
k12_school	Flexible	63.46	27.67	0.44
warehouse	Flexible	40.90	5.97	0.15
movie_theater	Non_Critical	32.65	4.50	0.14
hotel	Non_Critical	23.70	5.08	0.21
gas_station	Non_Critical	21.41	3.50	0.16
commercial_office	Flexible	14.17	2.42	0.17
restaurant	Non_Critical	13.71	6.22	0.45
fire_station	Critical	11.53	1.59	0.14
fitness_center	Non_Critical	11.51	1.60	0.14
library	Flexible	11.47	2.18	0.19
police_station	Critical	10.75	1.45	0.13
bus_station	Non_Critical	10.30	1.45	0.14
fast_food	Non_Critical	9.51	3.87	0.41
post_office	Non_Critical	3.83	1.14	0.30
clinic	Critical	1.79	0.40	0.22

```
In [6]: # Exercise 1.1 – Reading the data before modeling

# Q1: Look at the building_summary table above.
# Does mean energy demand alone separate the three tiers cleanly?
# Give a specific example where two buildings from different tiers
# have similar mean demand.
# A: No it does not, police station and bus station are next to eachother
```

```

# Q2: Look at the variability column (std/mean).
#   Which tier tends to have the LOWEST variability?
#   What does low variability tell you about how that type of building operates?
# A: The critical tier tends to have the lowest variability, this tells me it operates
    more efficiently.

# Q3: Police station and fire station are in the Critical tier
#   but have relatively LOW mean energy demand.
#   If energy demand alone doesn't make them critical, what does?
#   This is a domain knowledge question – not a data question.
# A: The need to the community does, they are first responders. In an emergency they
    respond quickly.

# Q4: Look at the histogram plots. Which tier has the widest spread of values?
#   What does that tell you about how hard it will be to classify that tier?
# A: The flexible tier does, this tells me that it will be hard to classify them.

# Q5: Before training any model – based only on what you can see in the data –
#   which buildings do you think the model will struggle to classify correctly?
#   Write your prediction here. You will check it after training.
# A: Shopping mall, college, clinic, police station, fire station

```

Part 2: Your Priority Tier Labels

The dataset comes with pre-assigned priority tiers. But your team defined your own priority tiers in Task 5 of Week 1 — before you saw any model results.

Now you need to reconcile those two things. Do you agree with the pre-assigned labels? If not, this is the moment to change them — with justification.

This is a genuine design decision. There is no single correct answer. Different teams will make different choices, and judges will ask you to defend yours.

Use your Quick Reference sheet — the power restoration priority order is the framework. But remember: that order was designed for a traditional grid. Your smart city might make different choices.

```

In [7]: # Current label assignments – review and modify if your team disagrees
# Format: 'building_type': 'tier'
# Tiers must be exactly: 'Critical', 'Flexible', or 'Non_Critical'

# REVIEW EACH ASSIGNMENT AND DECIDE IF YOU AGREE
# Write your reasoning for any changes in the comments below

priority_labels = {
    # Critical – power restored first, life-safety or essential infrastructure
    'hospital': 'Critical', # agree / disagree? reasoning: Agree, life s
    'clinic': 'Critical', # agree / disagree? reasoning: Agree, life s
    'police_station': 'Critical', # agree / disagree? reasoning: Agree, first
    'fire_station': 'Critical', # agree / disagree? reasoning: Agree, first

    # Flexible – power restored second, important but can tolerate delay

```

```

'college':          'Non_Critical',# agree / disagree? reasoning: Disagree, col
'k12_school':       'Flexible',    # agree / disagree? reasoning: Agree, acts a
'commercial_office': 'Non_Critical',# agree / disagree? reasoning: Disagree, bui
'library':          'Flexible',    # agree / disagree? reasoning: Agree, acts a
'warehouse':        'Non_Critical',# agree / disagree? reasoning: Disagree, bui

# Non_Critical – power restored last
'restaurant':       'Flexible',    # agree / disagree? reasoning: Disagree, foo
'hotel':            'Non_Critical',# agree / disagree? reasoning: Agree, not im
'grocery':          'Critical',    # agree / disagree? reasoning: Disagree, gro
'fast_food':        'Flexible',    # agree / disagree? reasoning: Disagree, che
'post_office':      'Non_Critical',# agree / disagree? reasoning: Agree, mail c
'shopping_mall':    'Non_Critical',# agree / disagree? reasoning: Agree, shoppi
'fitness_center':   'Non_Critical',# agree / disagree? reasoning: Agree, workin
'bus_station':      'Flexible',    # agree / disagree? reasoning: Disagree, tra
'movie_theater':    'Non_Critical',# agree / disagree? reasoning: Agree, entyer
'gas_station':      'Flexible',    # agree / disagree? reasoning: Disagree, nee
}

# Apply your labels to the dataset
df['priority_tier'] = df['building_type'].map(priority_labels)

print("Your priority tier assignments:")
print("-" * 45)
for tier in ['Critical', 'Flexible', 'Non_Critical']:
    buildings = [b for b, t in priority_labels.items() if t == tier]
    print(f"\n{tier} ({len(buildings)} buildings):")
    for b in buildings:
        print(f"    • {b}")

```

Your priority tier assignments:

Critical (5 buildings):

- hospital
- clinic
- police_station
- fire_station
- grocery

Flexible (6 buildings):

- k12_school
- library
- restaurant
- fast_food
- bus_station
- gas_station

Non_Critical (8 buildings):

- college
- commercial_office
- warehouse
- hotel
- post_office
- shopping_mall
- fitness_center
- movie_theater

```
In [8]: # Exercise 2.1 – Justify your most difficult labeling decisions

# Q1: Which building type was hardest to assign to a tier? Why?
#   Consider: grocery stores during a hurricane provide food for
#   emergency responders. Does that change their tier?
# A: Grocery store, food is crucial but is it as crucial as first responders

# Q2: A school becomes an emergency shelter during a hurricane.
#   Should that change its priority tier? How would your smart city
#   system handle a building whose priority changes depending on context?
# A: I took this into my current ranking, could make a system that takes the emerge

# Q3: Your team assigned 5 buildings to Critical tier.
#   During a severe outage, the grid can only support 30% of normal capacity.
#   If your Critical tier consumes more than 30% of grid capacity,
#   what happens? Was your Critical tier too large?
# A: They take up just under 30%, but if it did go over it could spread load based

# Q4: Who should make these labeling decisions in a real smart city system?
#   The AI engineer? The utility company? The city government? Citizens?
#   What are the implications of each?
# A: The government should, they make the emergency plans and will know what to do
```

Part 3: Feature Engineering for Classification

The features that matter for classification are different from regression. For regression, you wanted features that predicted *how much* demand would be. For classification, you want features that reveal *what kind* of building this is.

Think about what distinguishes a Critical building from a Non-Critical one:

- Critical buildings run at high demand **around the clock** — low variability
- Non-Critical buildings have **peaks and valleys** — high variability
- Flexible buildings are somewhere in between

Your features need to capture that behavioral signature.

```
In [9]: # Feature 1: variability_ratio (already in dataset as global_std_kWh / global_mean_kWh)
# Hospitals run 24/7 – low variability
# Restaurants peak at meal times – high variability
# This ratio captures operational pattern better than raw demand

df['variability_ratio'] = df['global_std_kWh'] / (df['global_mean_kWh'] + 1e-6)

# Verify – Critical buildings should have LOWER variability
print("Average variability ratio by tier:")
print(df.groupby('priority_tier')['variability_ratio'].mean().round(3))
print("\nDoes lower variability = more Critical? (should be yes)")
```

Average variability ratio by tier:

```
priority_tier
Critical      0.207
Flexible      0.298
Non_Critical  0.241
Name: variability_ratio, dtype: float64
```

Does lower variability = more Critical? (should be yes)

```
In [10]: # Feature 2: overnight_demand_ratio
# Critical buildings (hospitals) maintain HIGH demand overnight
# Non-critical buildings (restaurants, shops) go quiet at night
# This feature captures whether a building is truly 24/7 or business-hours only

# YOUR CODE: Create a column that is 1 if hour is between 22 and 6 (overnight), else 0
# Hint: use (df['hour'] >= 22) | (df['hour'] <= 6)
# then compare energy_kWh to global_mean_kWh during overnight hours

df['is_overnight'] = (df['hour'] >= 22) | (df['hour'] <= 6)

# Given: compute overnight demand ratio per building
overnight_mean = df[df['is_overnight'] == 1].groupby('building_type')['energy_kWh'].mean()
df['overnight_mean_kWh'] = df['building_type'].map(overnight_mean)
df['overnight_demand_ratio'] = df['overnight_mean_kWh'] / (df['global_mean_kWh'] + 1e-6)

# Justification (one sentence – why does overnight behavior help classify priority?)
#

print("Average overnight demand ratio by tier:")
print(df.groupby('priority_tier')['overnight_demand_ratio'].mean().round(3))
```

```
print("\nHospitals should have ratio near 1.0 (demand stays high overnight)")
print("Restaurants should have ratio well below 1.0 (quiet overnight)")
```

Average overnight demand ratio by tier:

priority_tier

Critical 0.841

Flexible 0.718

Non_Critical 0.772

Name: overnight_demand_ratio, dtype: float64

Hospitals should have ratio near 1.0 (demand stays high overnight)

Restaurants should have ratio well below 1.0 (quiet overnight)

```
In [11]: # Feature 3: peak_to_baseline_ratio
# How much does demand spike above its own baseline?
# Critical buildings have small spikes – they're always running hard
# Non-critical buildings have large spikes – busy periods vs quiet periods

# YOUR CODE: Create peak_to_baseline_ratio
# Formula: global_max_kWh divided by global_mean_kWh
# Example pattern: df['col'] = df['col_a'] / (df['col_b'] + 1e-6)

df['peak_to_baseline_ratio'] = df['global_max_kWh'] / (df['global_mean_kWh'] + 1e-6)

# Justification (one sentence):
# Divides global max over the global mean to calculate the baseline, the 1e-6 adds

print("Average peak-to-baseline ratio by tier:")
print(df.groupby('priority_tier')['peak_to_baseline_ratio'].mean().round(3))
print("\nLower ratio = more stable = more likely Critical")
```

Average peak-to-baseline ratio by tier:

priority_tier

Critical 1.369

Flexible 1.474

Non_Critical 1.417

Name: peak_to_baseline_ratio, dtype: float64

Lower ratio = more stable = more likely Critical

```
In [12]: # Exercise 3.1 – Design your own classification feature
# Create ONE feature that captures something the above features miss.
#
# Ideas:
# - business_hours_intensity: ratio of business-hours demand to overnight demand
# - demand_rank: rank each building by mean demand (1=highest)
# - is_24_7: binary flag for buildings that never drop below 80% of their mean
# - weekend_drop: how much demand falls on weekends vs weekdays
#
# Example patterns:
# df['col'] = df['col_a'] / df['col_b'] # ratio
# df['col'] = (df['col_a'] > threshold).astype(int) # binary flag
# df['col'] = df['col_a'].rank(pct=True) # percentile rank

# Feature name: (rename 'my_class_feature' to something descriptive)
# YOUR CODE:
df['demand_rank'] = (df.groupby('hour')['global_mean_kWh'].rank(ascending=False, me
```

```

# Why did you create this feature? What behavioral pattern does it capture?
# JUSTIFICATION: This feature will help determine how to spread power within each t
#

print("Demand rank by tier:")
print(df.groupby('priority_tier')['demand_rank'].mean().round(3))
print("\nDoes this feature separate the tiers? Does the direction make sense?")

```

```

Demand rank by tier:
priority_tier
Critical      20.504
Flexible      24.320
Non_Critical  16.635
Name: demand_rank, dtype: float64

```

Does this feature separate the tiers? Does the direction make sense?

```

In [13]: # Visualize your engineered features
# Do they actually separate the tiers visually?

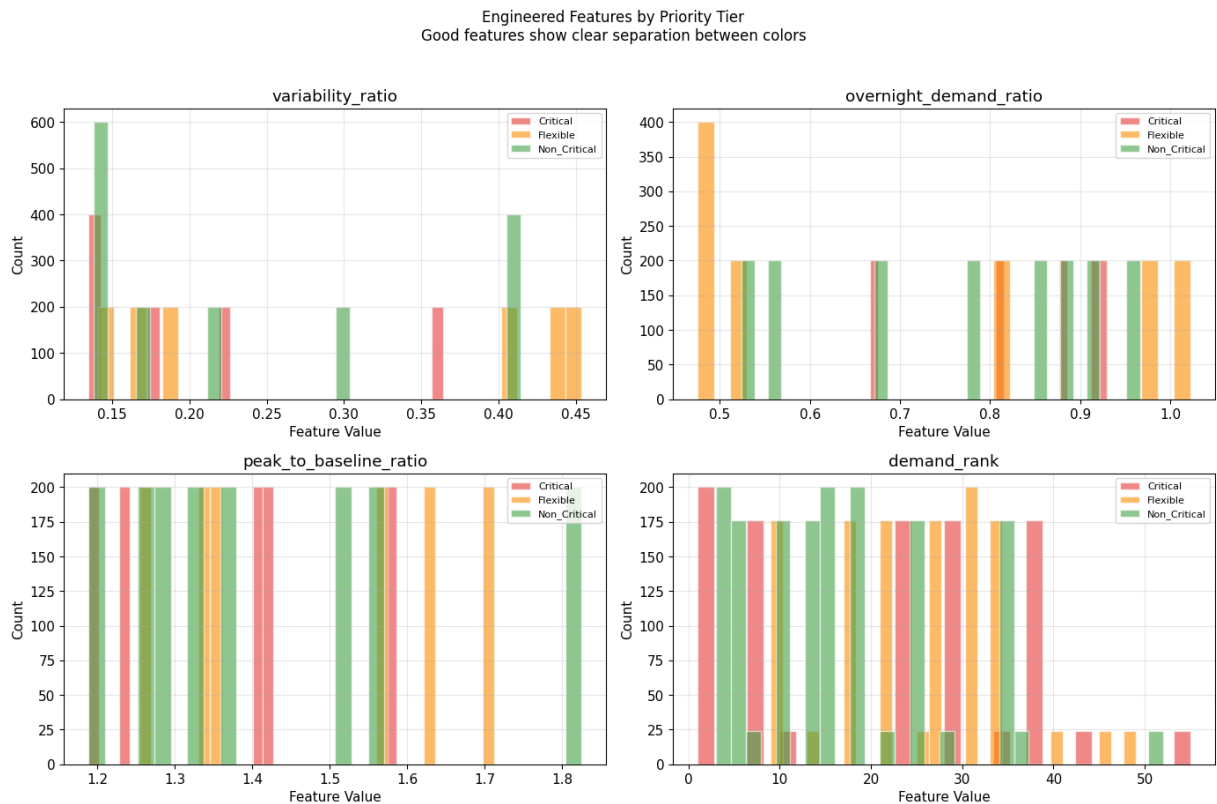
eng_features = ['variability_ratio', 'overnight_demand_ratio',
                'peak_to_baseline_ratio', 'demand_rank']

fig, axes = plt.subplots(2, 2, figsize=(14, 9))
axes = axes.flatten()

for i, feat in enumerate(eng_features):
    for tier, color in TIER_COLORS.items():
        tier_data = df[df['priority_tier'] == tier][feat].dropna()
        axes[i].hist(tier_data, bins=30, color=color, alpha=0.6,
                     edgecolor='white', label=tier)
    axes[i].set_title(feat)
    axes[i].set_xlabel('Feature Value')
    axes[i].set_ylabel('Count')
    axes[i].legend(fontsize=8)

plt.suptitle('Engineered Features by Priority Tier\n'
             'Good features show clear separation between colors',
             fontsize=12, y=1.02)
plt.tight_layout()
plt.show()

```



Part 4: Feature Selection and Model Training

A **decision tree** classifies by asking a series of yes/no questions about your features. At each branch it asks the question that best separates the classes — like a flowchart that the data taught itself.

Decision trees are interpretable — you can read the rules the model learned and ask *does this make sense?* That matters enormously when the model's decisions affect who gets power during an emergency.

```
In [14]: # Exercise 4.1 – YOUR FEATURE SELECTION
# Choose which features to include. You must include at least 4.
# Write your reasoning for each included AND excluded feature.
#
# Available features:
# 'energy_kWh', 'global_mean_kWh', 'global_max_kWh', 'global_std_kWh',
# 'hour', 'is_business_hours', 'pct_of_daily_max',
# 'variability_ratio', 'overnight_demand_ratio',
# 'peak_to_baseline_ratio', 'demand_rank'

# FEATURES WE ARE INCLUDING AND WHY:
# 1. global_mean_kWh      – because: Gives model the average
# 2. global_std_kWh       – because: Gives model average deviation
# 3. variability_ratio     – because: Gives model a sense of how much fluctuation is
# 4. hour                 – because: its the timestamp
# 5. energy_kWh           – because: Raw ewnergy data
```

```

# 6. is_business_hours - because: gives an boost during buisness hours
# 7. pct_of_daily_max - because: helps predict how much more is needed
# 8. demand_rank - becuase: helps spread power to individuals rather than t

# FEATURES WE ARE EXCLUDING AND WHY:
# 1. global_max_kWh - because: Taken care of by variability ratio
# 2. overnight_demand_ratio - because: taken care of by hour and business hours
# 3. peak_to_baseline_ratio - because: taken care of by the daily max

selected_features = [
    'global_mean_kWh',
    'global_std_kWh',
    'variability_ratio',
    'hour',
    'energy_kWh',
    'is_business_hours',
    'pct_of_daily_max',
    'demand_rank'
    # ADD MORE FEATURE NAMES HERE AS STRINGS
]

X = df[selected_features].fillna(0)
y = df['priority_tier']

print(f"Selected {len(selected_features)} features: {selected_features}")
print(f"X shape: {X.shape}")
print(f"\nClass distribution:")
print(y.value_counts())

```

Selected 8 features: ['global_mean_kWh', 'global_std_kWh', 'variability_ratio', 'hour', 'energy_kWh', 'is_business_hours', 'pct_of_daily_max', 'demand_rank']

X shape: (3800, 8)

Class distribution:
priority_tier
Non_Critical 1600
Flexible 1200
Critical 1000
Name: count, dtype: int64

```

In [15]: # Train / test split - stratified to preserve class balance
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"Training rows: {len(X_train):,}")
print(f"Test rows: {len(X_test):,}")
print(f"\nClass balance in test set:")
print(y_test.value_counts())
print(f"\nNote: stratify=y ensures each class is proportionally represented")
print(f"in both training and test sets. Without this, rare classes could")
print(f"be underrepresented in training - the model would never learn them.")

```

Training rows: 3,040
Test rows: 760

Class balance in test set:

```
priority_tier
Non_Critical    320
Flexible        240
Critical        200
Name: count, dtype: int64
```

Note: stratify=y ensures each class is proportionally represented in both training and test sets. Without this, rare classes could be underrepresented in training – the model would never learn them.

```
In [16]: # Exercise 4.2 – Choose your tree depth
# max_depth controls how many questions the tree can ask.
# Shallow trees (depth 2-3) are simple but may miss patterns.
# Deep trees (depth 8+) may memorize training data but fail on new data.
#
# Try at least TWO different depths and compare results.
# Choose the depth you will use for your final model and justify it.

# YOUR DEPTH EXPERIMENTS:
for depth in [3, 6, 8]:
    dt_temp = DecisionTreeClassifier(max_depth=depth, random_state=42)
    dt_temp.fit(X_train, y_train)
    train_acc = accuracy_score(y_train, dt_temp.predict(X_train))
    test_acc = accuracy_score(y_test, dt_temp.predict(X_test))
    print(f"max_depth={depth}: train={train_acc:.3f} test={test_acc:.3f} "
          f"gap={train_acc-test_acc:.3f}")

    print()
    print("A large gap between train and test accuracy suggests overfitting.")
    print("Your chosen depth and reasoning:")
    # CHOSEN DEPTH: 6
    # REASONING: 3 has too large a gap, and the gap of 0 for 8 seems way too suspicious.
```

```
max_depth=3: train=0.739 test=0.726 gap=0.013
max_depth=6: train=0.949 test=0.942 gap=0.007
max_depth=8: train=1.000 test=1.000 gap=0.000
```

A large gap between train and test accuracy suggests overfitting.
Your chosen depth and reasoning:

```
In [17]: # Train your final model
# Replace 5 with your chosen depth from the experiment above
CHOSEN_DEPTH = 6 # REPLACE WITH YOUR CHOSEN DEPTH

model = DecisionTreeClassifier(max_depth=CHOSEN_DEPTH, random_state=42)
model.fit(X_train, y_train)

print(f"Decision tree trained (max_depth={CHOSEN_DEPTH})")
print(f"\nWhat the model learned – feature importances:")
print("-" * 45)
for feat, imp in sorted(zip(selected_features, model.feature_importances_),
                        key=lambda x: x[1], reverse=True):
    bar = '█' * int(imp * 30)
```

```
print(f"{feat:<28} {imp:.3f} {bar}")
```

Decision tree trained (max_depth=6)

What the model learned – feature importances:

```
-----
variability_ratio      0.434 ██████████
global_mean_kWh        0.360 ████████
demand_rank            0.111 █████
global_std_kWh          0.096 ████
hour                   0.000
energy_kWh              0.000
is_business_hours       0.000
pct_of_daily_max        0.000
```

```
In [18]: # Read the decision tree rules
# This is what makes decision trees powerful for high-stakes decisions:
# you can read exactly what the model is doing and ask if it makes sense

tree_rules = export_text(model, feature_names=selected_features)
print("Decision Tree Rules:")
print("(Each branch is a yes/no question. Follow the path to get a classification.)")
print("-" * 60)
print(tree_rules)
```

Decision Tree Rules:

(Each branch is a yes/no question. Follow the path to get a classification.)

```

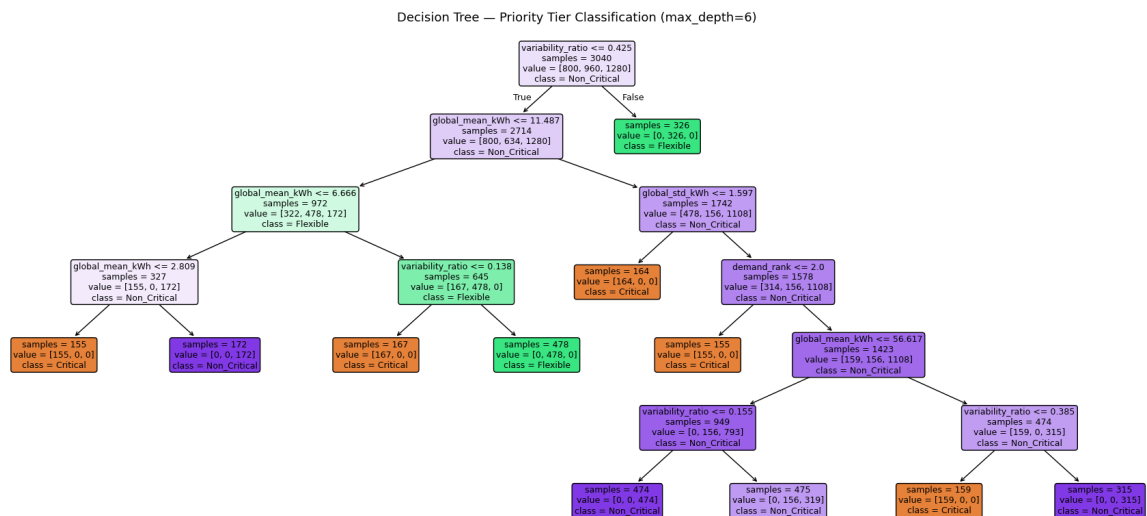
-----
|--- variability_ratio <= 0.43
|   |--- global_mean_kWh <= 11.49
|   |   |--- global_mean_kWh <= 6.67
|   |   |   |--- global_mean_kWh <= 2.81
|   |   |   |   |--- class: Critical
|   |   |   |   |--- global_mean_kWh > 2.81
|   |   |   |   |   |--- class: Non_Critical
|   |   |   |--- global_mean_kWh > 6.67
|   |   |   |   |--- variability_ratio <= 0.14
|   |   |   |   |   |--- class: Critical
|   |   |   |   |   |--- variability_ratio > 0.14
|   |   |   |   |   |   |--- class: Flexible
|   |   |--- global_mean_kWh > 11.49
|   |   |   |--- global_std_kWh <= 1.60
|   |   |   |   |--- class: Critical
|   |   |   |   |--- global_std_kWh > 1.60
|   |   |   |   |   |--- demand_rank <= 2.00
|   |   |   |   |   |   |--- class: Critical
|   |   |   |   |   |   |--- demand_rank > 2.00
|   |   |   |   |   |   |   |--- global_mean_kWh <= 56.62
|   |   |   |   |   |   |   |   |--- variability_ratio <= 0.15
|   |   |   |   |   |   |   |   |   |--- class: Non_Critical
|   |   |   |   |   |   |   |   |   |--- variability_ratio > 0.15
|   |   |   |   |   |   |   |   |   |   |--- class: Non_Critical
|   |   |   |   |   |--- global_mean_kWh > 56.62
|   |   |   |   |   |   |--- variability_ratio <= 0.38
|   |   |   |   |   |   |   |--- class: Critical
|   |   |   |   |   |   |   |--- variability_ratio > 0.38
|   |   |   |   |   |   |   |   |--- class: Non_Critical
|--- variability_ratio > 0.43
|   |--- class: Flexible

```

```

In [19]: # Visualize the tree – the painter's view of the model's Logic
fig, ax = plt.subplots(figsize=(18, 8))
plot_tree(
    model,
    feature_names=selected_features,
    class_names=model.classes_,
    filled=True,
    rounded=True,
    fontsize=9,
    ax=ax,
    impurity=False
)
ax.set_title(f'Decision Tree – Priority Tier Classification (max_depth={CHOSEN_DEPT}
             fontsize=13)
plt.tight_layout()
plt.show()

```

In [20]: # Exercise 4.3 – Reading the tree

```

# Q1: What is the FIRST question the tree asks (the root node)?
# Does that make intuitive sense as the most important distinguishing factor?
# A: Is the variability ratio less than or equal to 0.425, this makes sense as most

# Q2: Which feature has the highest importance score?
# Does that match your hypothesis from the Drawing with Data notebook?
# A: The variability ratio. no, I predicted hour would be the strongest

# Q3: Trace the path for a hospital:
# global_mean_kWh ≈ 492, global_std_kWh ≈ 88, variability_ratio ≈ 0.18
# Follow the tree rules above – which tier does the model assign it?
# Is that correct?
# A: It assigns it critical, which is correct

# Q4: Could you explain this tree's logic to a city council member
# who has no data science background? Try in 2-3 sentences.
# A: The tree starts by filtering out major flexible sources with the variability ratio
# It continues filtering by either mean_kWh, variability ratio, or the demand rank.
  
```

Part 5: Evaluation — The Confusion Matrix

For classification, the confusion matrix is more informative than a single accuracy number. It shows you **which classes get confused with which** — and in a power emergency, not all errors are equal.

- Classifying a **hospital as Non-Critical** = life-safety failure
- Classifying a **movie theater as Critical** = wasted scarce power
- Classifying a **school as Critical** instead of Flexible = minor overallocation

The confusion matrix lets you see which errors your model makes — and decide whether you can live with them.

```
In [21]: # Generate predictions
y_pred = model.predict(X_test)

# Overall accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Overall accuracy: {accuracy:.3f} ({accuracy*100:.1f}%)")
print(f"\nPer-class performance:")
print("-" * 50)
print(classification_report(y_test, y_pred,
                             target_names=['Critical', 'Flexible', 'Non_Critical']))
```

Overall accuracy: 0.942 (94.2%)

Per-class performance:

```
-----
              precision    recall  f1-score   support

   Critical           1.00        1.00        1.00         200
  Flexible           1.00        0.82        0.90         240
Non_Critical         0.88        1.00        0.94         320

 accuracy                   0.94         760
  macro avg              0.96        0.94        0.94         760
  weighted avg           0.95        0.94        0.94         760
```

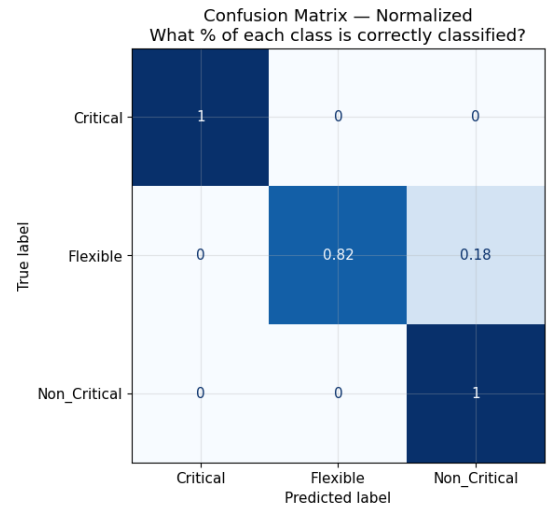
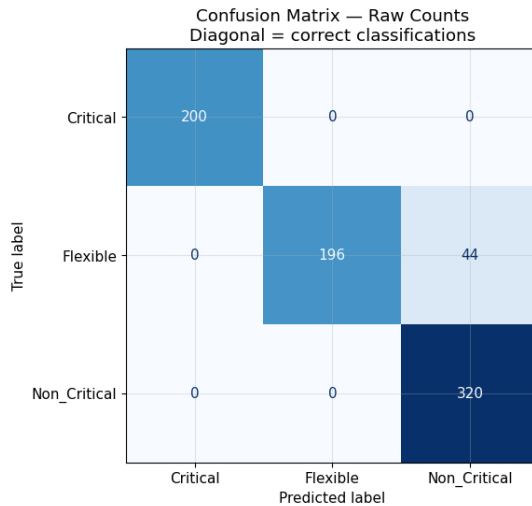
```
In [22]: # Plot the confusion matrix
fig, axes = plt.subplots(1, 2, figsize=(15, 6))

# Raw counts
cm = confusion_matrix(y_test, y_pred,
                      labels=['Critical', 'Flexible', 'Non_Critical'])
disp1 = ConfusionMatrixDisplay(confusion_matrix=cm,
                               display_labels=['Critical', 'Flexible', 'Non_Critical'])
disp1.plot(ax=axes[0], colorbar=False, cmap='Blues')
axes[0].set_title('Confusion Matrix - Raw Counts\n'
                  'Diagonal = correct classifications')

# Normalized (percentages)
cm_norm = confusion_matrix(y_test, y_pred,
                           labels=['Critical', 'Flexible', 'Non_Critical'],
                           normalize='true')
disp2 = ConfusionMatrixDisplay(confusion_matrix=cm_norm,
                               display_labels=['Critical', 'Flexible', 'Non_Critical'])
disp2.plot(ax=axes[1], colorbar=False, cmap='Blues')
axes[1].set_title('Confusion Matrix - Normalized\n'
                  'What % of each class is correctly classified?')

plt.suptitle('Classification Results by Priority Tier', fontsize=13, y=1.02)
plt.tight_layout()
plt.show()
```

Classification Results by Priority Tier



```
In [23]: # Which specific buildings are misclassified?
# This is the most important diagnostic for a high-stakes system

results = X_test.copy()
results['building_type'] = df.loc[X_test.index, 'building_type'].values
results['actual_tier'] = y_test.values
results['predicted_tier'] = y_pred
results['correct'] = results['actual_tier'] == results['predicted_tier']

misclassified = results[~results['correct']]

print("Misclassified buildings:")
print("-" * 55)
if len(misclassified) == 0:
    print("No misclassifications! (Check your features – this may indicate overfitt
else:
    summary = misclassified.groupby(
        ['building_type', 'actual_tier', 'predicted_tier']
    ).size().reset_index(name='count')
    summary['error_type'] = summary.apply(
        lambda r: '⚠️ DANGEROUS'
        if r['actual_tier'] == 'Critical' and r['predicted_tier'] != 'Critical'
        else '⚡ WASTEFUL'
        if r['actual_tier'] != 'Critical' and r['predicted_tier'] == 'Critical'
        else 'minor', axis=1
    )
    print(summary.to_string(index=False))
```

Misclassified buildings:

```
-----
building_type actual_tier predicted_tier count error_type
gas_station    Flexible    Non_Critical    44    minor
```

```
In [24]: # Exercise 5.1 – The confusion matrix as a risk assessment
```

```
# Q1: Look at the normalized confusion matrix.
# What percentage of Critical buildings were correctly classified?
# Is that acceptable for a life-safety system? What would be?
```

```

# A: 100% of critical buildings were classified correctly, this is perfectly acceptable.

# Q2: Are any Critical buildings being misclassified as Non_Critical?
#     (Look at the top-right cell of the confusion matrix.)
#     If yes – which building, and what would happen in a real emergency?
# A: Nope, not a single one

# Q3: The misclassified buildings table labels errors as DANGEROUS, WASTEFUL, or minor.
#     Which type of error does your model make most?
#     Which type is worse for the community?
# A: DANGEROUS is the worst, but my model only had minor, meaning that it performed well.

# Q4: Your model has overall accuracy of 94.2%. A city official asks:
#     'Is this model ready to deploy?' What would you tell them?
#     Use specific numbers from the confusion matrix, not just overall accuracy.
# A: yes, the model is ready to deploy, it only made confusions on the gas stations.
#     This is not a crucial error at all, a gas station requires nearly no power to operate.

# Q5: Go back to Exercise 1.1 Q5 – your prediction of which buildings
#     would be misclassified. Were you right? What does that tell you about
#     the value of drawing the data before modeling?
# A: No, I predicted that the buildings with high variability would cause the most misclassification.

```

Part 6: The Ethics of Misclassification

Every classification error in this model has a human consequence. This section asks you to think carefully about what those consequences are — and who bears them.

This is not a coding exercise. It is the most important part of the notebook.

```

In [25]: # Scenario: Your smart city experiences a major hurricane.
# Grid capacity is reduced to 40% of normal for 72 hours.
# Your classification model is directing power allocation automatically.

# Calculate approximate power demand by tier
critical_mean = df[df['priority_tier'] == 'Critical']['global_mean_kWh'].mean()
flexible_mean = df[df['priority_tier'] == 'Flexible']['global_mean_kWh'].mean()
noncrit_mean  = df[df['priority_tier'] == 'Non_Critical']['global_mean_kWh'].mean()

n_critical    = len([b for b, t in priority_labels.items() if t == 'Critical'])
n_flexible    = len([b for b, t in priority_labels.items() if t == 'Flexible'])
n_noncrit     = len([b for b, t in priority_labels.items() if t == 'Non_Critical'])

total_critical = critical_mean * n_critical
total_flexible = flexible_mean * n_flexible
total_all      = df['global_mean_kWh'].mean() * 19

print("Hurricane Scenario – Power Budget Analysis")
print("=" * 50)
print(f"Total normal grid demand:      {total_all:.0f} kWh")
print(f"Available at 40% capacity:      {total_all * 0.4:.0f} kWh")
print(f"")

```

```

print(f"Critical tier demand ({n_critical} buildings):  {total_critical:.0f} kWh")
print(f"Critical as % of available:  {total_critical/(total_all*0.4)*100:.1f}%")
print(f"")
print(f"Critical + Flexible demand:  {total_critical + total_flexible:.0f} kWh")
print(f"As % of available:  {(total_critical+total_flexible)/(total_all*0

```

Hurricane Scenario – Power Budget Analysis

=====

Total normal grid demand: 1412 kWh

Available at 40% capacity: 565 kWh

Critical tier demand (5 buildings): 589 kWh

Critical as % of available: 104.2%

Critical + Flexible demand: 718 kWh

As % of available: 127.2%

```

In [26]: # Exercise 6.1 – The hardest questions
        # Answer all five. These belong in your Week 4 design proposal.

        # Q1: Based on the power budget analysis above –
        #       can your Critical tier be fully powered at 40% grid capacity?
        #       If not, what does that mean for your classification system?
        #       Do you need to add a fourth tier? Reduce Critical tier size?
        # A: The critical tier could not be fully powered, it would be just over the aviala
        #       for example, a grocery store would not need to have all its lights on and many

        # Q2: Your model misclassified 1 building.
        #       For EACH misclassified building, describe the real-world consequence
        #       of that error during the 72-hour hurricane scenario.
        #       Be specific – who is affected and how?
        # A: People in need of gasoline are effected, gas stations would not have the nessa

        # Q3: Your model learned priority tiers from labels YOUR TEAM assigned.
        #       If a different team assigned different labels, they would get
        #       a different model with different errors.
        #       What does this tell you about the relationship between human decisions
        #       and AI outputs? Who is responsible for the model's errors?
        # A: People are responsible for a model's errors. The models are made and decided

        # Q4: During the hurricane, a grocery store is classified as Non_Critical
        #       and loses power. Emergency responders needed that store for food supplies.
        #       Was this the model's fault? The labeler's fault? The system designer's fault?
        #       What would you change?
        # A: This would be the system designers fault. Their system in the one that failed
        #       I would change the model, use differnt data inputs and setting in order to deve

        # Q5: Should an AI system be making these decisions automatically,
        #       or should a human review every classification before it affects power deliver
        #       What is the tradeoff between speed and human oversight in an emergency?
        # A: Human input is still crucial in an emergency, a human has the ability to criti

```

Part 7: Connecting to Optimization

You now have two models:

- A **regression model** that predicts how much power each building needs
- A **classification model** that assigns each building a priority tier

In Week 4 you will combine these into an **optimization model** that decides how to distribute finite power across your community — maximizing benefit while respecting the priority order your classification defined.

The final reflection prepares you for that work.

```
In [27]: # Final Reflection – answer all four questions
# These answers belong in your Week 4 design proposal.

# Q1: Your classification model achieved 94.2% accuracy.
# Your regression model achieved  $R^2 = 0.958$ .
# Which model are you more confident in for emergency deployment?
# Explain using specific metrics – not just which number is higher.
# A: Im more confident in using the classification model, the classification model
# only failing 18% of the time on gas stations, which are flexible. This leaves m
# get critical infrastructure right, almost if not all of the time.

# Q2: The decision tree asks questions about energy consumption patterns
# to classify priority. But priority is fundamentally a human value judgment,
# not a data pattern.
# What is the model actually learning – the right answer, or YOUR answer?
# Is there a difference?
# A: The model learns my answer, it is trained based of my selection, and it theref
# There iss a difference, no human has abasolutly perfect judgement, infact a rig

# Q3: Name one type of building that your classification system cannot handle well.
# (Think about buildings whose priority changes depending on context –
# a school during a hurricane vs a normal day.)
# How would you modify the system to handle context-dependent priority?
# A: A school changes based on context, and my model classifies it as flexible no m

# Q4: You have now drawn your data, built a regression model, and built
# a classification model. You started with a blank canvas.
# Looking at everything your team has built – what is the most important
# thing you learned that you did NOT expect to learn?
# A: Models find things you won't, I previosly believed that ML models just quantif
```

Extension: Clustering — What If You Had No Labels?

(Optional — required for teams competing for top marks)

Your classification model learned from labels YOUR TEAM assigned. But what if no one had decided on priority tiers ahead of time?

Clustering is an unsupervised technique — the algorithm finds natural groupings in the data

WITHOUT being told what the groups should be. You then interpret what each cluster means.

Use K-Means clustering with $k=3$ on the same features you used for classification. Compare the clusters the algorithm found to your human-assigned tiers.

Key questions:

- Did the algorithm find groups that match your priority tiers?
- Where did it disagree? What does that tell you?
- Which approach — human labels or data-driven clusters — would you trust more for a real emergency system? Why?

Write a 3-4 sentence analysis for your Week 4 design proposal.

```
In [28]: # Extension – Clustering
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# YOUR CODE HERE
# 1. Scale your features (KMeans is sensitive to scale)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
#
# 2. Fit KMeans with k=3
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
kmeans.fit(X_scaled)
#
# 3. Compare cluster assignments to your priority labels
# Look at which building types land in each cluster
df['cluster'] = kmeans.labels_

print('Cluster assignments by building type:')
print(df.groupby(['building_type', 'priority_tier'])['cluster'].first().sort_values)

# 4. Visualize using a scatter plot of two features, colored by cluster
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 6))

for cluster in range(3):
    mask = kmeans.labels_ == cluster
    ax1.scatter(X_scaled[mask, selected_features.index('global_mean_kWh')],
                X_scaled[mask, selected_features.index('variability_ratio')],
                label=f'Cluster {cluster}', alpha=0.6, s=50)
ax1.set_title('Algorithm Clusters')
ax1.legend()

for tier, color in TIER_COLORS.items():
    mask = df.loc[X.index, 'priority_tier'] == tier
    ax2.scatter(X_scaled[mask, selected_features.index('global_mean_kWh')],
                X_scaled[mask, selected_features.index('variability_ratio')],
                label=tier, alpha=0.6, s=50, color=color)
ax2.set_title('Your Manual Priority Tiers')
ax2.legend()
```

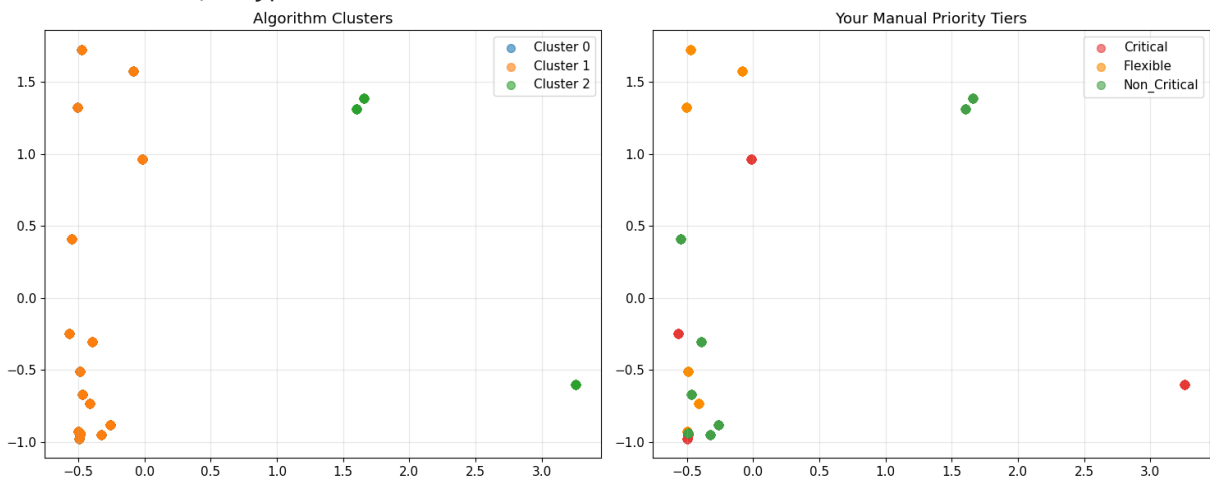
```
plt.tight_layout()
plt.show()
```

*# YOUR ANALYSIS: The cluster model was completely unable to create correct groupings
 # The cluster model completely ignored one cluster, and only assigned the three high
 # When looking at a graph of the two most important features it is easy to see why
 # The selections from the human labels appear to be scattered completely randomly, an*

Cluster assignments by building type:

building_type	priority_tier	
bus_station	Flexible	1
clinic	Critical	1
commercial_office	Non_Critical	1
fast_food	Flexible	1
fitness_center	Non_Critical	1
fire_station	Critical	1
gas_station	Flexible	1
grocery	Critical	1
police_station	Critical	1
hotel	Non_Critical	1
k12_school	Flexible	1
library	Flexible	1
post_office	Non_Critical	1
movie_theater	Non_Critical	1
warehouse	Non_Critical	1
restaurant	Flexible	1
hospital	Critical	2
college	Non_Critical	2
shopping_mall	Non_Critical	2

Name: cluster, dtype: int32



What Comes Next

You have now drawn the data, predicted demand with regression, and classified users by priority. The three colors on your palette are mixed.

In Week 4 you will paint the finished picture: an optimization model that distributes finite

power across your smart city community — using everything your team has built, debated, and learned.

Your regression predictions tell you how much each building needs. Your classification tiers tell you who gets served first. Your optimization model decides how to make those two things work together when the grid cannot give everyone what they need.

Quick Start Orange County: AI Design Contest — University of Florida